Executions · Lamar

GIU Curriculum and Academic Innovation Management Platform (GCAIMP) · Milestone 2 · executions divided between the team, same split as Procedures.pdf

From the Milestone 2 document (GIU, Winter 2026, submission 15/11/2026, 11:59 pm):

d) **Executions.sql**: an example execution of every stored procedure and function (**EXEC** for procedures, **SELECT** for functions), using data from your insertions.

• The four SQL files must run in this order without errors: Tables.sql, Insertions.sql, Procedures.sql, Executions.sql.

• It is the students' responsibility to make sure that every stored procedure and function executes correctly and returns the expected output.

• Your procedures and functions must handle any errors or exceptions that may happen (e.g., invalid input, records that do not exist, duplicate records, or violated constraints) and show a clear message instead of failing.

How we do it: write your calls in `executions/<YourName>.sql` on your branch, one execution per procedure or function below, in the same order. Use data from Insertions.sql. Add a second call that triggers an error case where it makes sense. Merge into main and the CI glues it into Executions.sql.

Lamar 24 executions (one per procedure / function)

1 As a System Admin, I should be able to:

13. Add a new program to a faculty.
    Signature:
    Name: AddProgram.
    Type: Stored procedure.
    Input: @Name varchar(150), @DegreeLevel varchar(50), @TotalCreditHours int, @StandardDuration varchar(50), @Description varchar(1000), @FacultyID int.
    Output: Confirmation message or new @ProgramID int.

14. Add a new course as either a Core or an Elective course.
    Signature:
    Name: AddCourse.
    Type: Stored procedure.
    Input: @CourseCode varchar(20), @Title varchar(150), @Description varchar(1000), @CreditHours int, @Level varchar(50), @CourseType varchar(20), @DefaultStudySemester int, @RequiredForProgram varchar(150), @ElectivePool varchar(150), @MinSelectionRule varchar(150).
    Output: Confirmation message.

15. Link a course to a prerequisite or co-requisite course.
    Signature:
    Name: AddCourseRequisite.
    Type: Stored procedure.
    Input: @CourseCode varchar(20), @RequisiteCode varchar(20), @RequisiteType varchar(50).
    Output: Confirmation message.

2 As an Academic Instructor, I should be able to:

2. Add an intended learning outcome (ILO) to a course.

Signature:
Name: AddILO.
Type: Stored procedure.
Input: @CourseCode varchar(20), @ILOCode varchar(20), @Statement varchar(1000), @CognitiveLevel varchar(50).
Output: Confirmation message.

3. Update the statement or cognitive level of an existing ILO.
   Signature:
   Name: UpdateILO.
   Type: Stored procedure.
   Input: @CourseCode varchar(20), @ILOCode varchar(20), @Statement varchar(1000), @CognitiveLevel varchar(50).
   Output: Confirmation message.

4. Map an ILO of my course to a program learning outcome (PLO).
   Signature:
   Name: MapILOToPLO.
   Type: Stored procedure.
   Input: @CourseCode varchar(20), @ILOCode varchar(20), @ProgramID int, @PLOCode varchar(20), @CoverageLevel varchar(50), @IsAssessmentPoint bit.
   Output: Confirmation message or new @MappingID int.

8. Submit a curriculum modification proposal, optionally linked to the assessment that motivated it.
   Signature:
   Name: SubmitModificationProposal.
   Type: Stored procedure.
   Input: @ProposerID int, @Title varchar(150), @Rationale varchar(1000), @ModificationType varchar(50), @CurrentStateSummary varchar(1000), @ProposedStateSummary varchar(1000), @CourseCode varchar(20), @ProgramID int, @VersionLabel varchar(20), @AcademicYear varchar(50), @AssessmentID int.
   Output: Confirmation message or new @ProposalID int.

9. Revise a proposal that was returned for revision.
   Signature:
   Name: ReviseModificationProposal.
   Type: Stored procedure.
   Input: @ProposalID int, @Rationale varchar(1000), @ProposedStateSummary varchar(1000).
   Output: Confirmation message.

10. View the full approval history of one of my proposals.
    Signature:
    Name: ViewProposalStatus.
    Type: Function.
    Input: @ProposalID int.
    Output: Table showing decision, decision level, approver, decision date and comments.

11. View the courses of a curriculum version grouped by study semester.
    Signature:
    Name: ViewCurriculumVersion.
    Type: Function.
    Input: @ProgramID int, @VersionLabel varchar(20), @AcademicYear varchar(50).

Output: Table showing study semester, course code, title, credit hours and whether it is mandatory.

12. Compare two curriculum versions of the same program.
    Signature:
    Name: CompareCurriculumVersions.
    Type: Function.
    Input: @ProgramID int, @OldVersionLabel varchar(20), @OldAcademicYear varchar(50), @NewVersionLabel varchar(20), @NewAcademicYear varchar(50).
    Output: Table of courses that were added, removed or moved to another study semester.

3 As a Teaching Assistant, I should be able to:

3. View the ILOs of a course I support.
    Signature:
    Name: ViewCourseILOs.
    Type: Function.
    Input: @CourseCode varchar(20).
    Output: Table of ILO codes, statements and cognitive levels.

4 As a Curriculum Reviewer, I should be able to:

1. View the full curriculum map of a program (which ILOs map to which PLOs).
    Signature:
    Name: ViewProgramCurriculumMap.
    Type: Function.
    Input: @ProgramID int.
    Output: Table showing PLO, course, ILO, coverage level and assessment point.

2. Identify PLOs that are not covered by any approved mapping.
    Signature:
    Name: GetUncoveredPLOs.
    Type: Function.
    Input: @ProgramID int.
    Output: Table of uncovered PLO codes and statements.

3. View how often each PLO is introduced, reinforced and mastered across courses.
    Signature:
    Name: GetPLOCoverageSummary.
    Type: Function.
    Input: @ProgramID int.
    Output: Table showing PLO, number of courses at each coverage level and number of assessment points.

4. Approve or reject an outcome mapping.
    Signature:
    Name: ReviewOutcomeMapping.
    Type: Stored procedure.
    Input: @MappingID int, @MappingStatus varchar(50).
    Output: Confirmation message.

8. View pending modification proposals for a program.
    Signature:
    Name: ViewPendingProposals.
    Type: Function.

Input: @ProgramID int.
Output: Table of proposals with no final decision, with proposer and submission date.

9. Record my review decision on a modification proposal.
   Signature:
   Name: ReviewModificationProposal.
   Type: Stored procedure.
   Input: @ProposalID int, @ReviewerID int, @Decision varchar(50), @Comments varchar(1000).
   Output: Confirmation message.

10. View the history of proposals and decisions for a course.
    Signature:
    Name: ViewCourseProposalHistory.
    Type: Function.
    Input: @CourseCode varchar(20).
    Output: Table of proposals with modification type, submission date and latest decision.

5 As a Dean, I should be able to:

2. Approve, reject, or return a modification proposal for revision.
   Signature:
   Name: DecideOnProposal.
   Type: Stored procedure.
   Input: @ProposalID int, @DeanID int, @Decision varchar(50), @Comments varchar(1000).
   Output: Confirmation message.

5. Add a program learning outcome (PLO) to a program in my faculty.
   Signature:
   Name: AddPLO.
   Type: Stored procedure.
   Input: @ProgramID int, @PLOCode varchar(20), @Statement varchar(1000), @Category varchar(50).
   Output: Confirmation message.

6. Create a new draft curriculum version, optionally copying the course placements of an existing version.
   Signature:
   Name: CreateCurriculumVersion.
   Type: Stored procedure.
   Input: @ProgramID int, @VersionLabel varchar(20), @AcademicYear varchar(50), @EffectivePeriod varchar(50), @CopyFromVersionLabel varchar(20), @CopyFromAcademicYear varchar(50).
   Output: Confirmation message.

7. Place a course in a curriculum version at a specific study semester.
   Signature:
   Name: PlaceCourseInVersion.
   Type: Stored procedure.
   Input: @ProgramID int, @VersionLabel varchar(20), @AcademicYear varchar(50), @CourseCode varchar(20), @StudySemesterNo int, @IsMandatory bit.
   Output: Confirmation message.

8. Approve a curriculum version, making it Active and archiving the previously active version.
   Signature:
   Name: ApproveCurriculumVersion.
   Type: Stored procedure.

Input: @ProgramID int, @VersionLabel varchar(20), @AcademicYear varchar(50).
Output: Confirmation message.